

## Week- 6 Skill

1.To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
imvarls@LAPTOP-VQ4D70L7: × + ∨
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

int main() {
    char history[MAX_HISTORY][MAX_COMMAND];
    int count = 0;
    char command[MAX_COMMAND];

    while (1) {
        printf("history-shell> ");
        if (fgets(command, sizeof(command), stdin) == NULL) break;
        command[strcspn(command, "\n")] = 0;

        if (strcmp(command, "exit") == 0) {
            break;
        }

        if (strcmp(command, "history") == 0) {
            printf("\nCommand History:\n");
            for (int i = 0; i < count; i++) {
                printf("%d: %s\n", i + 1, history[i]);
            }
            continue;
        }

        if (count < MAX_HISTORY) {
            strcpy(history[count], command);
            count++;
        } else {
            for (int i = 1; i < MAX_HISTORY; i++) {
                strcpy(history[i - 1], history[i]);
            }
            strcpy(history[MAX_HISTORY - 1], command);
        }
        printf("Command stored.\n");
    }

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + ∨
imvarls@LAPTOP-VQ4D70L7:~$ nano command_history.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc command_history.c -o command_history
imvarls@LAPTOP-VQ4D70L7:~$ ./command_history
history-shell> n
Command stored.
history-shell> ls
Command stored.
history-shell> pwd
Command stored.
history-shell> date
Command stored.
history-shell> whoami
Command stored.
history-shell> history

Command History:
1: n
2: ls
3: pwd
4: date
5: whoami
history-shell> |
```

2. To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

int main() {
    char input[500];
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int count = 0;

    printf("Enter pipeline: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = 0;

    char *token = strtok(input, "|");

    while (token != NULL && count < MAX_COMMANDS) {
        while (*token == ' ') {
            token++;
        }

        strcpy(commands[count], token);
        count++;

        token = strtok(NULL, "|");
    }

    printf("\nPipeline Layout:\n");
    for (int i = 0; i < count; i++) {
        printf("Command %d: %s\n", i + 1, commands[i]);
    }

    printf("\nNumber of commands: %d\n", count);
    printf("Number of pipes: %d\n", count - 1);

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: ~$ nano pipeline.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc pipeline.c -o pipeline
imvarls@LAPTOP-VQ4D70L7: ~$ ./pipeline
Enter pipeline: ls | grep.c |sort

Pipeline Layout:
Command 1: ls
Command 2: grep.c
Command 3: sort

Number of commands: 3
Number of pipes: 2
imvarls@LAPTOP-VQ4D70L7: ~$ |
```

### 3. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    pipe(pipefd);

    pid1 = fork();

    if (pid1 == 0) {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("ls", "ls", NULL);
        perror("ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 == 0) {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        execlp("grep", "grep", ".c", NULL);
        perror("grep");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline completed.\n");

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: ~$ nano singlepipe.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc singlepipe.c -o singlepipe
imvarls@LAPTOP-VQ4D70L7: ~$ ./singlepipe
File1.txt.c
client.c
command_history.c
copy.c
copy_file.c
demo.c
directory
directory.c
export.c
file.c
file2.c
filename.c
filename.c.save
first.c.save
first.c.save.1
myshell.c
navigation.c
pipeline.c
process
process.c
process_state
process_state.c
processstate.c
server.c
shell.c
singlepipe.c
task2.c
week1.c
week4.c
week5.c
week5task2.c
week6client
week6client.c
week6server.c
week6task2.c
zombie.c
Pipeline completed.
imvarls@LAPTOP-VQ4D70L7: ~$ |
```

#### 4.To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char *commands[][4] = {
        {"ls", NULL},
        {"grep", ".c", NULL},
        {"sort", NULL}
    };
    int n = 3;
    int pipes[2][2];
    pid_t pids[3];

    for (int i = 0; i < n - 1; i++) {
        pipe(pipes[i]);
    }

    for (int i = 0; i < n; i++) {
        pids[i] = fork();
        if (pids[i] == 0) {
            if (i > 0) {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }
            if (i < n - 1) {
                dup2(pipes[i][1], STDOUT_FILENO);
            }
            for (int j = 0; j < n - 1; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }
            execvp(commands[i][0], commands[i]);
            perror("execvp");
            exit(1);
        }
    }

    for (int i = 0; i < n - 1; i++) {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < n; i++) {
        waitpid(pids[i], NULL, 0);
    }

    printf("Long pipeline completed.\n");
    return 0;
}
```

Output:



imvarls@LAPTOP-VQ4D70L7: X



```
imvarls@LAPTOP-VQ4D70L7:~$ nano multiple_pipe.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc multiple_pipe.c -o multiple_pipe
imvarls@LAPTOP-VQ4D70L7:~$ ./multiple_pipe
File1.txt.c
client.c
command_history.c
copy.c
copy_file.c
demo.c
directory
directory.c
export.c
file.c
file2.c
filename.c
filename.c.save
first.c.save
first.c.save.1
multiple_pipe.c
myshell.c
navigation.c
pipeline.c
process
process.c
process_state
process_state.c
processstate.c
server.c
shell.c
singlepipe.c
task2.c
week1.c
week4.c
week5.c
week5task2.c
week6client
week6client.c
week6server.c
week6task2.c
zombie.c
Long pipeline completed.
imvarls@LAPTOP-VQ4D70L7:~$ |
```